

Práctica 9

Manejo de servo con retroalimentación posicional

Grado en Ingeniería en Robótica Software

GSyC, Universidad Rey Juan Carlos



©2024 Julio Vega Pérez

©2025 Esther Aguado

Algunos derechos reservados.

Este trabajo se entrega bajo licencia CC-BY-SA 4.0.

1. Introducción

En esta práctica vamos a trabajar con un servo continuo; esto es, se trata de un motor que gira vueltas completas, en un sentido y en otro, pero además nos puede dar realimentación de la posición que tiene, de ahí su denominación de servo.

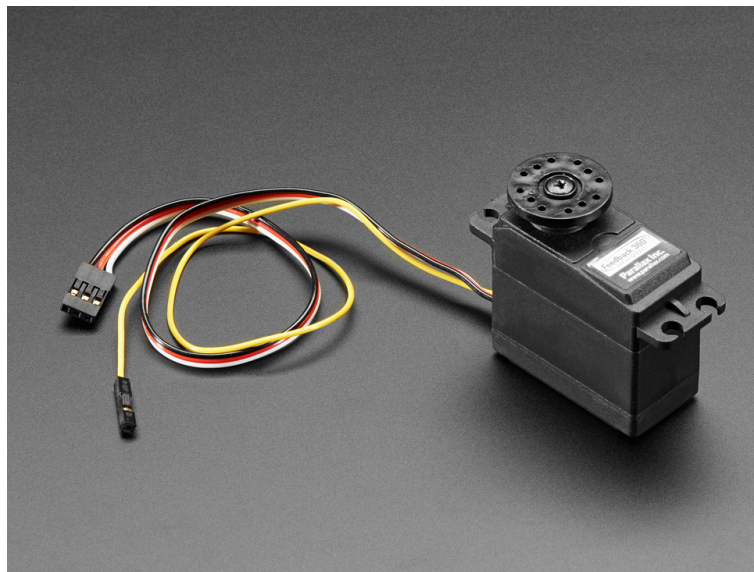


Figura 1: Modelo Feedback 360 High Speed Servo de Parallax

Un problema habitual con los servos continuos que podemos encontrar en cualquier tienda de electrónica es que generalmente no tienen rotación de 360 grados y además retroalimentación posicional. Uno de los pocos servos que podemos encontrar en el mercado que cuente con ambas cualidades y que sea económico (*DIY*) es el *Feedback 360 High Speed Servo* (Figura 1), de la compañía *Parallax*, y es con el que vamos a trabajar nosotros.

Otra compañía que también ofrece algo similar, y que además es muy importante en el mundo *DIY*, es *Dynamixel*, con su económico modelo *AX-12A*. Pero tengamos en cuenta que la mayoría de servos no

disponen de esta retroalimentación posicional.

2. Mecanismo interno: sensor de efecto Hall

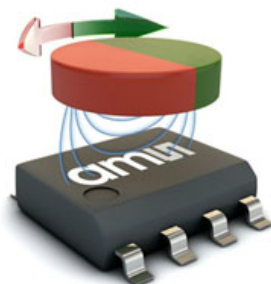


Figura 2: Sensor AS5035 de efecto Hall sin contacto

El modelo de *Parallax* es básicamente un servo continuo normal que utiliza un sensor de los que ya hemos estudiado; concretamente, un sensor de efecto Hall sin contacto (modelo AS5600, Figura 2), que proporciona retroalimentación posicional como un tren de pulsos.

3. Funcionamiento y conexión del servo

Para poder programar el servo, y como siempre, lo primero es conocer cómo funciona, y para ello lo mejor es leerse las especificaciones (adjuntas a esta práctica). De estas podemos extraer información muy útil, por ejemplo los siguientes aspectos:

- Pin de señal de retroalimentación de posición: cable amarillo.
- Periodo de envío de señal de control: 20 ms.
- Ancho de pulso de la señal: p.ej. $1280 - 1480\mu s$ para girar en un sentido.
- Periodo de señal de retroalimentación (t_{Cycle}): $\frac{1}{910} Hz (\simeq 1.1 \text{ ms})$.
- Ciclo de trabajo de t_{Cycle} : $2,9 - 91,7\%$

El esquema de conexión se puede visualizar en la Figura 3.

4. Para practicar

Se adjunta el código `servo.ino`, que permite manipular el servo para que se mueva en ambas direcciones a máxima velocidad. Para ello, se utiliza la librería `Servo.h`, la cual proporciona funciones clave para el control del servo. Entre ellas, `attach()` permite enlazar un pin a un objeto tipo servo, mientras que `writeMicroseconds()` permite escribir un valor en microsegundos para controlar el servo (ver el datasheet para los valores recomendados). Para más información, revisa la documentación oficial de la librería en: Arduino Servo Library.

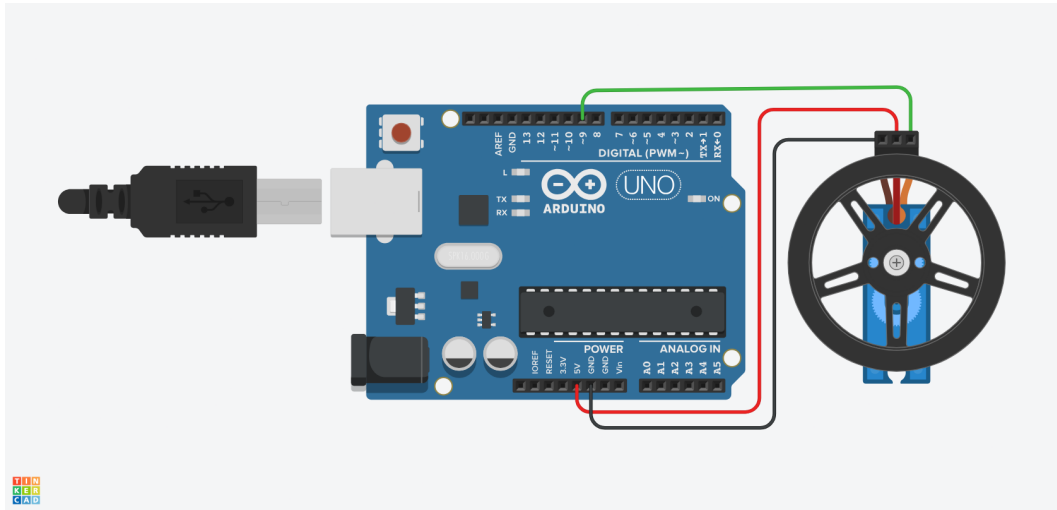


Figura 3: Esquema de conexión del servo

Ejercicios

Ejercicio 1

En este primer ejercicio se pide calibrar el servo para que gire en ambos sentidos a distintas velocidades. Esto implica encontrar los valores específicos que permitan controlar su movimiento, ya que estos pueden variar ligeramente entre diferentes servos. Por ello, es necesario realizar pruebas hasta identificar los valores adecuados para el servo disponible.

Ejercicio 2

Partiendo de lo logrado en el ejercicio anterior, ahora se busca encontrar una fórmula sencilla que relacione la velocidad (en m/s) con el comando del ciclo de trabajo necesario para alcanzarla. El usuario podrá enviar un comando entre -100 y 100 para controlar la velocidad y el sentido del giro del servo. La fórmula deberá ajustar este comando, superando la zona muerta del servo y manteniendo una relación proporcional con la entrada. Además, el signo del comando determinará la dirección del giro. Para leer números decimales con signo, se puede utilizar la función `Serial.parseInt()` o `Serial.parseFloat()`.

Ejercicio 3

Por último, vamos a intentar obtener la lectura posicional del servo, aunque sea de forma aproximada. Para ello, comandaremos una velocidad durante un corto periodo de tiempo para después medir cómo ha variado su posición angular. Para ello, debemos procesar la información del pin de *Feedback* (cable amarillo). En las especificaciones podemos encontrar información detallada de cómo trabaja este sistema, concretamente en la sección *Position Control System*. Para determinar el ciclo de trabajo, puedes utilizar la función `PulseIn()`, que mide la duración de un pulso en microsegundos, ya sea en estado ALTO o BAJO.

Entrega de la práctica

La entrega se realizará en el repositorio individual de GitHub Classroom. Cada repositorio debe contener al menos el código producido así como un archivo `README.md` que incluya la memoria de la práctica con los siguientes elementos:

La memoria de prácticas debe contener los siguientes elementos:

- Nombre de los estudiantes.
- Objetivo de la práctica.
- Resumen del trabajo realizado.
- Particularidades: Problemas encontrados, funcionalidad extra, etc.
- Vídeo del funcionamiento del ejercicio.
- Esquemático del circuito (Fritzing, Tinkercad, etc.).